JavaScript is disabled on your browser.

compact1, compact2, compact3

java.text

Class Collator

java.lang.Object
    java.text.Collator

All Implemented Interfaces:

`Cloneable, Comparator<Object>`

Direct Known Subclasses:

`RuleBasedCollator`

```
public abstract class Collator
extends Object
implements Comparator<Object>, Cloneable
```

The `Collator` class performs locale-sensitive `String` comparison. You use this class to build searching and sorting routines for natural language text.

`Collator` is an abstract base class. Subclasses implement specific collation strategies. One subclass, `RuleBasedCollator`, is currently provided with the Java Platform and is applicable to a wide set of languages. Other subclasses may be created to handle more specialized needs.

Like other locale-sensitive classes, you can use the static factory method, `getInstance`, to obtain the appropriate `Collator` object for a given locale. You will only need to look at the subclasses of `Collator` if you need to understand the details of a particular collation strategy or if you need to modify that strategy.

The following example shows how to compare two strings using the `Collator` for the default locale.

```
// Compare two strings in the default locale
Collator myCollator = Collator.getInstance();
if( myCollator.compare("abc", "ABC") < 0 )
```

```
        System.out.println("abc is less than ABC");
    else
        System.out.println("abc is greater than or equal to ABC");
```

You can set a `Collator`'s *strength* property to determine the level of difference considered significant in comparisons. Four strengths are provided: `PRIMARY`, `SECONDARY`, `TERTIARY`, and `IDENTICAL`. The exact assignment of strengths to language features is locale dependant. For example, in Czech, "e" and "f" are considered primary differences, while "e" and "ě" are secondary differences, "e" and "E" are tertiary differences and "e" and "e" are identical. The following shows how both case and accents could be ignored for US English.

```
//Get the Collator for US English and set its strength to PRIMARY
Collator usCollator = Collator.getInstance(Locale.US);
usCollator.setStrength(Collator.PRIMARY);
if( usCollator.compare("abc", "ABC") == 0 ) {
    System.out.println("Strings are equivalent");
}
```

For comparing `Strings` exactly once, the `compare` method provides the best performance. When sorting a list of `Strings` however, it is generally necessary to compare each `String` multiple times. In this case, `CollationKeys` provide better performance. The `CollationKey` class converts a `String` to a series of bits that can be compared bitwise against other `CollationKeys`. A `CollationKey` is created by a `Collator` object for a given `String`.
Note: `CollationKeys` from different `Collators` can not be compared. See the class description for `CollationKey` for an example using `CollationKeys`.

Implementation Note:

Significant thread contention may occur during concurrent usage of the JDK Reference Implementation's `RuleBasedCollator`, which is the subtype returned by the default provider of the `getInstance()` factory methods. As such, users should consider retrieving a separate instance for each thread when used in multithreaded environments.

See Also:

`RuleBasedCollator`, `CollationKey`, `CollationElementIterator`, `Locale`

Field Summary

Fields

Modifier and Type	Field	Description
static int	**CANONICAL_DECOMPOSITION**	Decomposition mode value.
static int	**FULL_DECOMPOSITION**	Decomposition mode value.

static int	IDENTICAL	Collator strength value.
static int	NO_DECOMPOSITION	Decomposition mode value.
static int	PRIMARY	Collator strength value.
static int	SECONDARY	Collator strength value.
static int	TERTIARY	Collator strength value.

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	Collator()	Default constructor.

Method Summary

Abstract Methods

Concrete Methods

Modifier and Type	Method	Description
Object	clone()	Overrides Cloneable
int	compare(Object o1, Object o2)	Compares its two arguments for order.
abstract int	compare(String source, String target)	Compares the source string to the target string according to the collation rules for this Collator.
boolean	equals(Object that)	Compares the equality of two Collators.

boolean	**equals**(**String** source, **String** target)	Convenience method for comparing the equality of two strings based on this Collator's collation rules.
static **Locale**[]	**getAvailableLocales**()	Returns an array of all locales for which the `getInstance` methods of this class can return localized instances.
abstract **CollationKey**	**getCollationKey**(**String** source)	Transforms the String into a series of bits that can be compared bitwise to other CollationKeys.
int	**getDecomposition**()	Get the decomposition mode of this Collator.
static **Collator**	**getInstance**()	Gets the Collator for the current default locale.
static **Collator**	**getInstance**(**Locale** desiredLocale)	Gets the Collator for the desired locale.
int	**getStrength**()	Returns this Collator's

		strength property.
abstract int	**hashCode**()	Generates the hash code for this Collator.
void	**setDecomposition**(int decompositionMode)	Set the decomposition mode of this Collator.
void	**setStrength**(int newStrength)	Sets this Collator's strength property.

Methods inherited from class java.lang.Object

finalize, getClass, notify, notifyAll, toString, wait, wait, wait

Methods inherited from interface java.util.Comparator

comparing, comparing, comparingDouble, comparingInt, comparingLong, naturalOrder, nullsFirst, nullsLast, reversed, reverseOrder, thenComparing, thenComparing, thenComparing, thenComparingDouble, thenComparingInt, thenComparingLong

Field Detail

PRIMARY

```
public static final int PRIMARY
```

Collator strength value. When set, only PRIMARY differences are considered significant during comparison. The assignment of strengths to language features is locale dependant. A common example is for different base letters ("a" vs "b") to be considered a PRIMARY difference.

See Also:

setStrength(int), getStrength(), Constant Field Values

SECONDARY

```
public static final int SECONDARY
```

Collator strength value. When set, only SECONDARY and above differences are considered significant during comparison. The assignment of strengths to language features is locale dependant. A common example is for different accented forms of the same base letter ("a" vs "&#65533;") to be considered a SECONDARY difference.

See Also:

`setStrength(int)`, `getStrength()`, `Constant Field Values`

TERTIARY

```
public static final int TERTIARY
```

Collator strength value. When set, only TERTIARY and above differences are considered significant during comparison. The assignment of strengths to language features is locale dependant. A common example is for case differences ("a" vs "A") to be considered a TERTIARY difference.

See Also:

`setStrength(int)`, `getStrength()`, `Constant Field Values`

IDENTICAL

```
public static final int IDENTICAL
```

Collator strength value. When set, all differences are considered significant during comparison. The assignment of strengths to language features is locale dependant. A common example is for control characters ("\u0001" vs "\u0002") to be considered equal at the PRIMARY, SECONDARY, and TERTIARY levels but different at the IDENTICAL level. Additionally, differences between pre-composed accents such as "\u00C0" (A-grave) and combining accents such as "A\u0300" (A, combining-grave) will be considered significant at the IDENTICAL level if decomposition is set to NO_DECOMPOSITION.

See Also:

`Constant Field Values`

NO_DECOMPOSITION

```
public static final int NO_DECOMPOSITION
```

Decomposition mode value. With NO_DECOMPOSITION set, accented characters will not be decomposed for collation. This is the default setting and provides the fastest collation but will only produce correct results for languages that do not use accents.

See Also:

`getDecomposition()`, `setDecomposition(int)`, Constant Field Values

CANONICAL_DECOMPOSITION

```
public static final int CANONICAL_DECOMPOSITION
```

Decomposition mode value. With CANONICAL_DECOMPOSITION set, characters that are canonical variants according to Unicode standard will be decomposed for collation. This should be used to get correct collation of accented characters.

CANONICAL_DECOMPOSITION corresponds to Normalization Form D as described in Unicode Technical Report #15.

See Also:

`getDecomposition()`, `setDecomposition(int)`, Constant Field Values

FULL_DECOMPOSITION

```
public static final int FULL_DECOMPOSITION
```

Decomposition mode value. With FULL_DECOMPOSITION set, both Unicode canonical variants and Unicode compatibility variants will be decomposed for collation. This causes not only accented characters to be collated, but also characters that have special formats to be collated with their norminal form. For example, the half-width and full-width ASCII and Katakana characters are then collated together. FULL_DECOMPOSITION is the most complete and therefore the slowest decomposition mode.

FULL_DECOMPOSITION corresponds to Normalization Form KD as described in Unicode Technical Report #15.

See Also:

`getDecomposition()`, `setDecomposition(int)`, Constant Field Values

Constructor Detail

Collator

```
protected Collator()
```

Default constructor. This constructor is protected so subclasses can get access to it. Users typically create a Collator sub-class by calling the factory method getInstance.

See Also:

`getInstance()`

Method Detail

getInstance

```
public static Collator getInstance()
```

Gets the Collator for the current default locale. The default locale is determined by java.util.Locale.getDefault.

Returns:

`the Collator for the default locale.(for example, en_US)`

See Also:

`Locale.getDefault()`

getInstance

```
public static Collator getInstance(Locale desiredLocale)
```

Gets the Collator for the desired locale.

Parameters:

`desiredLocale - the desired locale.`

Returns:

`the Collator for the desired locale.`

See Also:

`Locale`, `ResourceBundle`

compare

```
public abstract int compare(String source,
                            String target)
```

Compares the source string to the target string according to the collation rules for this Collator. Returns an integer less than, equal to or greater than zero depending on whether the source String is less than, equal to or greater

than the target string. See the Collator class description for an example of use.

For a one time comparison, this method has the best performance. If a given String will be involved in multiple comparisons, CollationKey.compareTo has the best performance. See the Collator class description for an example using CollationKeys.

Parameters:

`source - the source string.`

`target - the target string.`

Returns:

`Returns an integer value. Value is less than zero if source is less than target, value is zero if source and target are equal, value is greater than zero if source is greater than target.`

See Also:

`CollationKey`, `getCollationKey(java.lang.String)`

compare

```
public int compare(Object o1,
                   Object o2)
```

Compares its two arguments for order. Returns a negative integer, zero, or a positive integer as the first argument is less than, equal to, or greater than the second.

This implementation merely returns `compare((String)o1, (String)o2)` .

Specified by:

`compare` in interface `Comparator<Object>`

Parameters:

`o1` - the first object to be compared.

`o2` - the second object to be compared.

Returns:

a negative integer, zero, or a positive integer as the first argument is less than, equal to, or greater than the second.

Throws:

`ClassCastException` - the arguments cannot be cast to Strings.

Since:

1.2

See Also:

`Comparator`

getCollationKey

```
public abstract CollationKey getCollationKey(String source)
```

Transforms the String into a series of bits that can be compared bitwise to other CollationKeys. CollationKeys provide better performance than Collator.compare when Strings are involved in multiple comparisons. See the Collator class description for an example using CollationKeys.

Parameters:

`source - the string to be transformed into a collation key.`

Returns:

`the CollationKey for the given String based on this Collator's collation rules. If the source String is null, a null CollationKey is returned.`

See Also:

`CollationKey, compare(java.lang.String, java.lang.String)`

equals

```
public boolean equals(String source,
                      String target)
```

Convenience method for comparing the equality of two strings based on this Collator's collation rules.

Parameters:

`source - the source string to be compared with.`

`target - the target string to be compared with.`

Returns:

`true if the strings are equal according to the collation rules. false, otherwise.`

See Also:

`compare(java.lang.String, java.lang.String)`

getStrength

```
public int getStrength()
```

Returns this Collator's strength property. The strength property determines the minimum level of difference considered significant during comparison. See the Collator class description for an example of use.

Returns:

this Collator's current strength property.

See Also:

setStrength(int), PRIMARY, SECONDARY, TERTIARY, IDENTICAL

setStrength

```
public void setStrength(int newStrength)
```

Sets this Collator's strength property. The strength property determines the minimum level of difference considered significant during comparison. See the Collator class description for an example of use.

Parameters:

newStrength - the new strength value.

Throws:

IllegalArgumentException - If the new strength value is not one of PRIMARY, SECONDARY, TERTIARY or IDENTICAL.

See Also:

getStrength(), PRIMARY, SECONDARY, TERTIARY, IDENTICAL

getDecomposition

```
public int getDecomposition()
```

Get the decomposition mode of this Collator. Decomposition mode determines how Unicode composed characters are handled. Adjusting decomposition mode allows the user to select between faster and more complete collation behavior.

The three values for decomposition mode are:

- NO_DECOMPOSITION,
- CANONICAL_DECOMPOSITION
- FULL_DECOMPOSITION.

See the documentation for these three constants for a description of their meaning.

Returns:

the decomposition mode

See Also:

setDecomposition(int), NO_DECOMPOSITION, CANONICAL_DECOMPOSITION, FULL_DECOMPOSITION

setDecomposition

```
public void setDecomposition(int decompositionMode)
```

Set the decomposition mode of this Collator. See getDecomposition for a description of decomposition mode.

Parameters:

`decompositionMode - the new decomposition mode.`

Throws:

`IllegalArgumentException - If the given value is not a valid decomposition mode.`

See Also:

`getDecomposition()`, `NO_DECOMPOSITION`, `CANONICAL_DECOMPOSITION`, `FULL_DECOMPOSITION`

getAvailableLocales

```
public static Locale[] getAvailableLocales()
```

Returns an array of all locales for which the `getInstance` methods of this class can return localized instances. The returned array represents the union of locales supported by the Java runtime and by installed `CollatorProvider` implementations. It must contain at least a Locale instance equal to `Locale.US`.

Returns:

`An array of locales for which localized Collator instances are available.`

clone

```
public Object clone()
```

Overrides Cloneable

Overrides:

`clone` in class `Object`

Returns:

`a clone of this instance.`

See Also:

`Cloneable`

equals

```
public boolean equals(Object that)
```

Compares the equality of two Collators.

Specified by:

`equals` in interface `Comparator<Object>`

Overrides:

`equals` in class `Object`

Parameters:

`that - the Collator to be compared with this.`

Returns:

`true if this Collator is the same as that Collator; false otherwise.`

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public abstract int hashCode()
```

Generates the hash code for this Collator.

Overrides:

`hashCode` in class `Object`

Returns:

`a hash code value for this object.`

See Also:

```
Object.equals(java.lang.Object),
System.identityHashCode(java.lang.Object)
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.